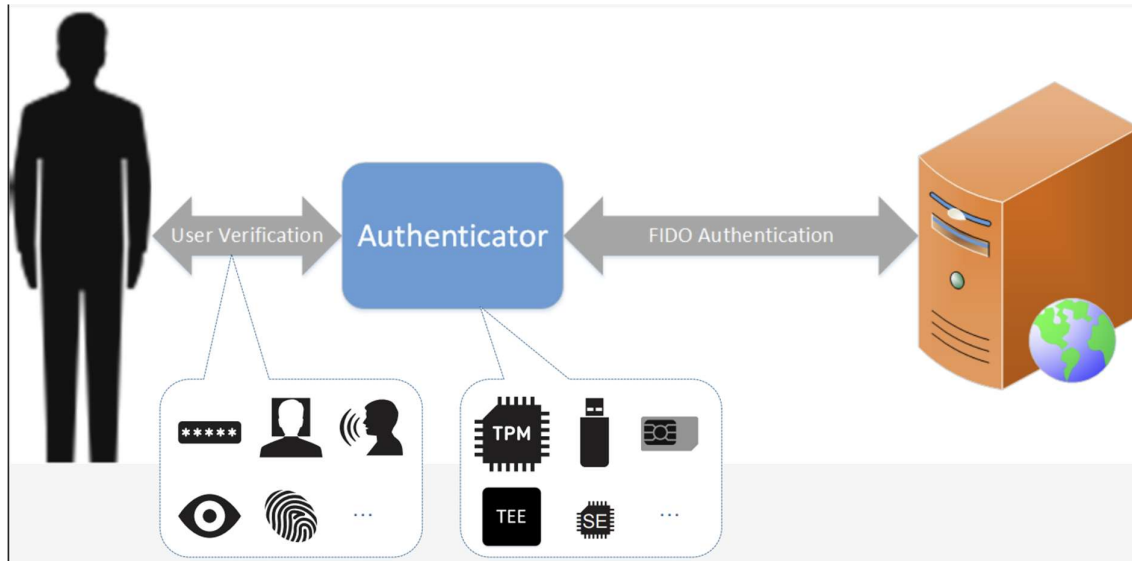


Fido Provider – Developer Guide

The fido provider is a simplified adaption of a Fido U2F client library functionality, as an alternative to the Optional Fido.dll binary library supplied within this GitHub repository.



Develop Fido Provider

To implement a software-based **FIDO U2F (CTAP1) client provider** in C#, you need to emulate how a physical security key behaves.

The core responsibility of a U2F client/authenticator is to respond to a server's authentication request by signing a challenge using **EdDsa (with the ed25519 /Edward's curve)**.

Supporting Routines

The following two functions are fully aligned to the Fido U2F standard

```
private byte[] DeriveKey(byte[] appId, byte[] keyHandle)
{
    // Extract
    byte[] nonce = keyHandle.Take(32).ToArray();
    byte[] _deviceKey = GetDeviceKey();
    using (var hmac256 = new HMACSHA256(_deviceKey)) // seed SHA256 with device key
    {
        // PrivateKey=HMAC(AppID+nonce,DeviceKey)
        MemoryStream pKeyBuff = new MemoryStream();
        BinaryWriter pKeyWriter = new BinaryWriter(pKeyBuff);
        pKeyWriter.Write(appId);
        pKeyWriter.Write(nonce);
        // privateKeyBytes is essentially a deterministic, random 32 byte array
        byte[] privateKeyBytes = hmac256.ComputeHash(pKeyBuff.ToArray());
        return privateKeyBytes;
    }
}

private byte[] DeriveKeyHandle(byte[] nonce, byte[] appId, byte[] privateKeyBytes)
{
    //KeyHandle=nonce,HMAC(AppID,PrivateKey)
    MemoryStream keyHandleBuff = new MemoryStream();
    BinaryWriter keyHandleWriter = new BinaryWriter(keyHandleBuff);
    // Write keyHandle ->nonce
    keyHandleWriter.Write(nonce);
    // Compute HMAC(AppID,PrivateKey)
    MemoryStream macBuff = new MemoryStream();
    BinaryWriter macWriter = new BinaryWriter(macBuff);
    macWriter.Write(nonce); // Attest binding to nonce, not in fido spec (add Aug 2023)
    macWriter.Write(appId); // AppID->HSM secure identity
    macWriter.Write(privateKeyBytes); // EcDsa IEEE PrivateKey
    // Write keyHandle -> HMAC(AppID,PrivateKey)

    byte[] _deviceKey = GetDeviceKey();
    var hmac256 = new HMACSHA256(_deviceKey); // preloaded device key
    // Compute Signature or keyHandle attestation data
    var sign = hmac256.ComputeHash(macBuff.ToArray());
    keyHandleWriter.Write(sign);
    //KeyHandle=nonce,HMAC(AppID,PrivateKey)
    byte[] KeyHandle = keyHandleBuff.ToArray();
    return KeyHandle;
}
```

3 references

```
public void Enrol(string type)
{
    // Determine technology type
    string relativeUrl;
    if (GetTechType() == TechType.Classic)
        relativeUrl = "User/Register";
    else
        relativeUrl = "api/Register";

    // Extract Function endpoint from User Token
    Uri _baseUri = new Uri(Audience());
    // Obtain Function JwtToken
    HttpClient _httpClient = new HttpClient();
    Uri _uri = new Uri(_baseUri, relativeUrl);
    _httpClient.BaseAddress = _baseUri;
    // Add Auth JwtToken
    string authToken = GetJwtToken();
    _httpClient.DefaultRequestHeaders.Add("x-token", authToken);
    _httpClient.DefaultRequestHeaders.Add("x-jwk", GetJwk());
    _httpClient.DefaultRequestHeaders.Add("x-xjwk", GetXJwk());
    string ssign = Sign(GetDeviceSin());
    _httpClient.DefaultRequestHeaders.Add("x-jws-signature", ssign);
    // Get response
    HttpResponseMessage response = _httpClient.GetAsync(_uri).Result;
    if (response.IsSuccessStatusCode)
    {
        TokenRoleType t = GetRoleType();
        // pass
        string json = response.Content.ReadAsStringAsync().Result;
        byte[] bjson = HostDeviceUnProtect(json);
        RecoveryResponse r = JsonSerializer.Deserialize<RecoveryResponse>(bjson);
        SetCounter((uint)r.counter);
        SetJwtToken(r.jwtToken);
        SetDomain(r.domain); // Optional Domain
        if (string.IsNullOrEmpty(r.pin) == false)
            SetPin(r.pin);
    }
    else
    {
        // fail
        string error = response.Content.ReadAsStringAsync().Result;
        throw new Exception(error);
    }
}
```

```
/// <summary>
/// Sign and return Signature (bytes[]),
/// </summary>
/// <param name="keyHandle"></param>
/// <param name="hashData"></param>
/// <returns></returns>
/// <exception cref="Exception"></exception>
```

22 references | 0/15 passing

```
public string Sign(string data)
{
    // Ed25519
    byte[] signature = _edDsa.SignData(data);
    return Base64Url.Encode(signature);
}
```

Within the Fido standard references to URL are replaced by Secure Identity end points. The only requirement is that these be unique with the scope of the infrastructure.

```
public static string DeviceIdentity(byte[] publickey)
{
    // Build public key bytes[]
    return "0199" + Convert.ToHexString(RIPEMD160.Create().ComputeHash(publickey)).ToLower();
}

public static string ExportEd25519ToJwk(byte[] publickey)
{
    // Build JsonWebKey
    MemoryStream ms = new MemoryStream();
    using (Utf8JsonWriter writer = new Utf8JsonWriter(ms))
    {
        writer.WriteStartObject();
        writer.WritePropertyName("Alg");
        writer.WriteStringValue("Ed25519");
        writer.WritePropertyName("Kty");
        writer.WriteStringValue("OKP");
        writer.WritePropertyName("Kid");
        writer.WriteStringValue(Convert.ToHexString(RIPEMD160.Create().ComputeHash(publickey)));
        writer.WritePropertyName("Crv");
        writer.WriteStringValue("Curve25519");
        writer.WritePropertyName("X");
        writer.WriteStringValue(Base64Url.Encode(publickey));
        writer.WriteEndObject();
    }
    return Encoding.UTF8.GetString(ms.ToArray());
}
```

Device Initial Registration

1. Obtain device Authorisation token from operator.
2. Execute Registration request, using the Local Fido Datastore and the supporting enrol code above.

```
/// <summary>
/// Register Device
/// </summary>
/// <param name="sin">Secure Identity</param>
/// <returns>KeyHandle</returns>
/// <exception cref="Exception"></exception>
1 reference
private void Register(ref DeviceStore ds)
{
    // FIDO AppId->SecureIdentity
    byte[] Appid = Encoding.UTF8.GetBytes(ds.SecureIdentity);
    // Generate registration nounce
    byte[] nounce = CsPrng.SecureRandom(32).ToArray();
    // Derive private key .
    byte[] _keyBytes = DeriveKey(Appid, nounce);
    byte[] KeyHandle = DeriveKeyHandle(nounce, Appid, _keyBytes);
    // Ed25519 signatures
    _edDsa = new EdDsa(_keyBytes);
    // Update DeviceStore
    ds.KeyHandle = KeyHandle;
    ds.JwKey = ExportEd25519ToJwk(_edDsa.PublicKey);
    ds.DeviceSin = DeviceIdentity(_edDsa.PublicKey);
}
```

3. Open the device to process API calls over TLS.